

Jimma University

Faculty of Computing and Informatics

Computer Science Department

Web Programming

Jimma, Ethiopia.

Chapter 3: Introduction to CSS

- A CSS allows you to separate website HTML content from its style. **We use HTML files to arrange the content**
  - All of the **presentation/formatting,** like fonts, colors, background, borders, text formatting, link effects, etc., are **accomplished with CSS**
- Using CSS, you can control the color of the text, the style of fonts, the spacing between paragraphs, how columns are sized and laid out, what background images or colors are used, as well as a variety of other effects.
- CSS is a web page layout method that has been added to HTML to **give web developers more control over their design and content layout.**
  - **Using CSS allows a designer to create a standard set of commands that controls the style of all subsequent pages.**
- These commands are embedded inside the web page or from an external file/page.

Cont...

- With **CSS you can add style** (fonts, colors, spacing, and size) to web documents.
  - More advanced techniques **control the layout of the page without the use of tables or other HTML tags.**
- **CSS separates the layout and the styles of a web page.**
  - Styles such as fonts color, font sizes, and margins, can be specified in one place, and then the Web pages feed off this one master list, with the styles cascading throughout the page or an entire site.
  - To alter the spacing of your content, split it into multiple columns, or add animations and other decorative features.

Cont...

- In HTML 5.0, all formatting should be removed from the HTML document, and stored in a separate CSS file.
- CSS is a **breakthrough in web design** because it **allows developers to control the style and layout of multiple web pages all at once.**
- As a web developer, you can define a style for each HTML element and apply it to as many web pages as you want.
  - To make a global change, simply change the style, and all elements in the Web are updated automatically.

Benefits of CSS

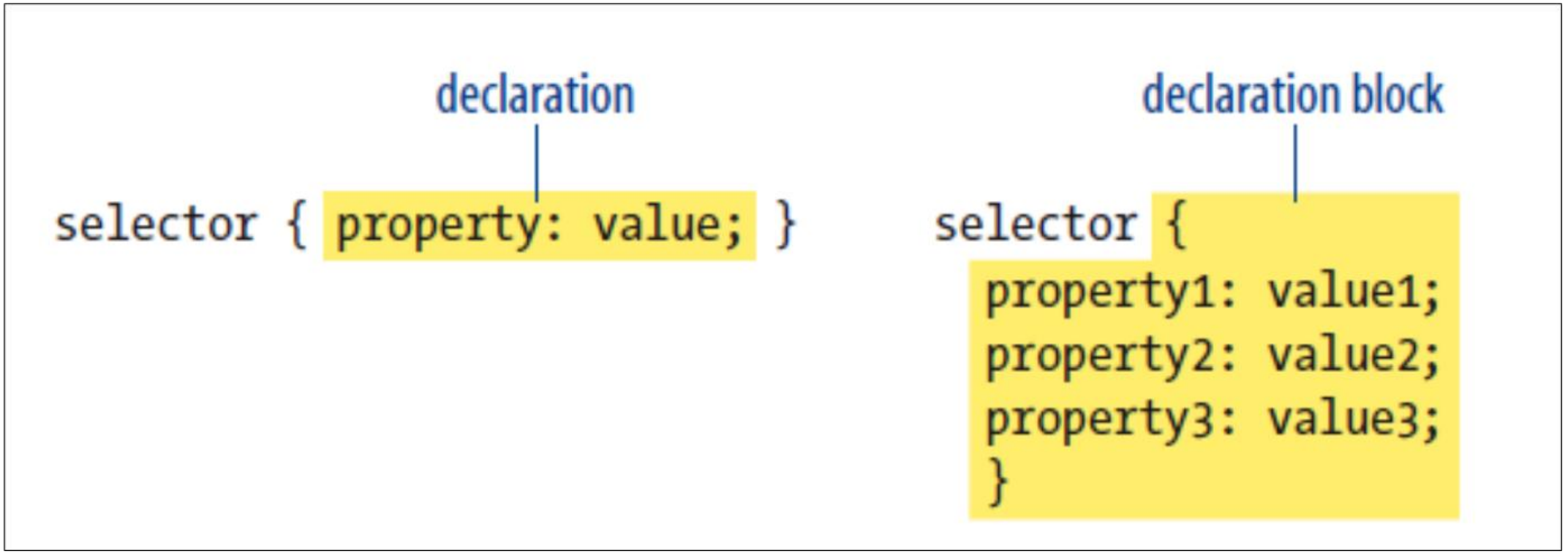
- **Better type and layout controls—**Presentational HTML never gets close to offering the kind of control over type, backgrounds, and layout, but it is possible with CSS.
- **Less work—**You can change the appearance of an entire site by editing one style sheet.
  - Making small tweaks and even entire site redesigns with style sheets is much easier than when presentation instructions are mixed in with the markup.
- **Potentially smaller documents and faster downloads—**Old school practices of using **redundant font elements** and **nested tables** make for bloated documents.
  - Not only that, you can apply a **single style sheet document** to all the pages in a site for further **byte savings**.
- **More accessible sites—**When all matters of presentation are handled by CSS, you can mark up your content meaningfully, making it more accessible for non-visual or mobile devices.

CSS Syntax

- A CSS rule has two main parts:
  - a selector, and
  - one or more declarations.
- The **selector** is normally the HTML element you want to style. It is an HTML tag at which a style will be applied.
  - Usually, it is the HTML tags to be styled. E.g. <p>, <h1>, <table>
- **Each declaration** consists of:
  - Property: A property is a type of attribute of HTML tag. Put simply, all the HTML attributes are converted into CSS properties. They could be color, border, etc.
  - A value. Values are assigned to properties. For example, color property can have the value either red or #F1F1F1 etc

Cont...

❖CSS declarations always end with a semicolon.

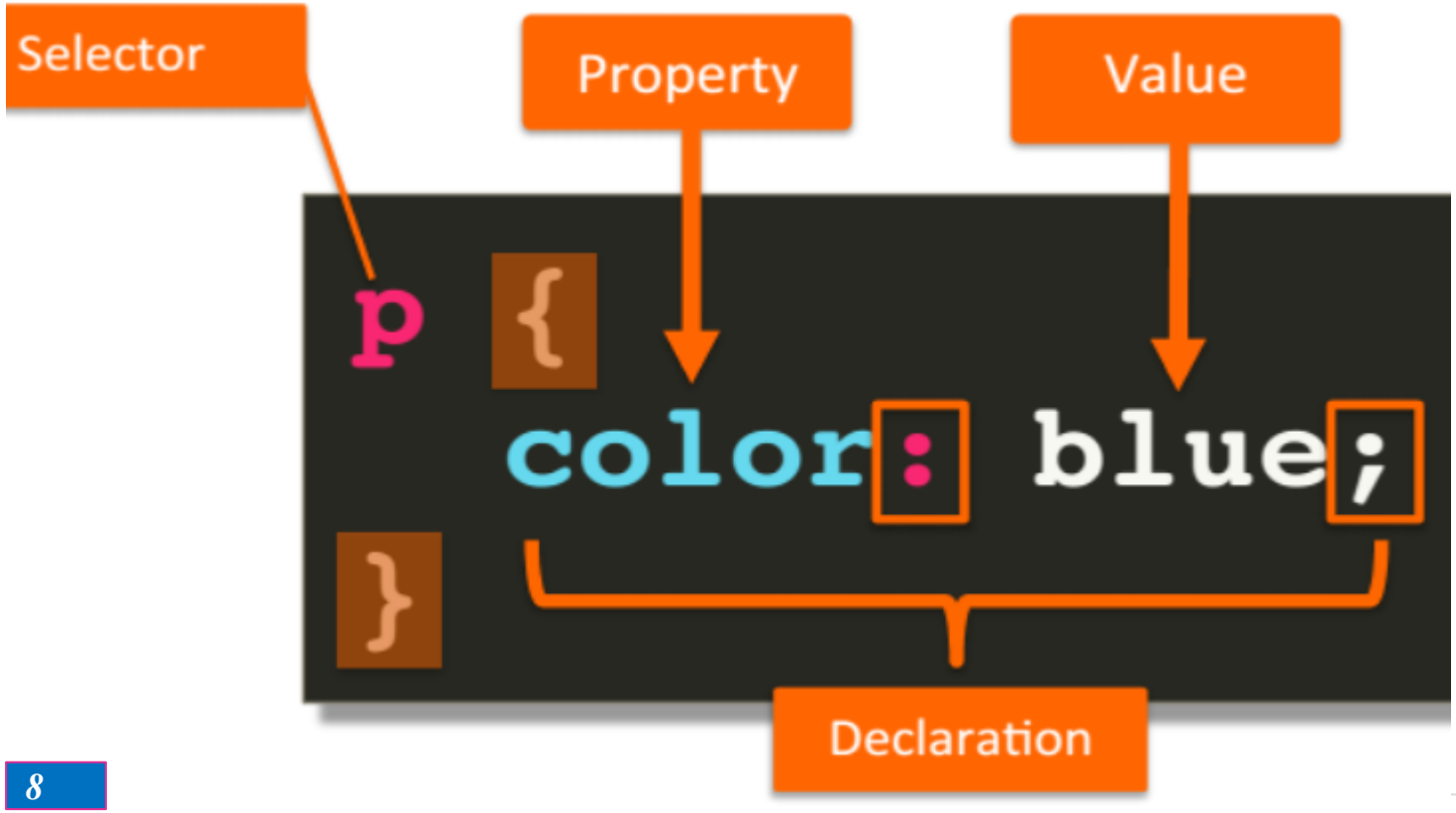


❖Declaration groups are surrounded by curly brace{ }

```
p {color: red; text-align: center;}
```

Cont...

- A style sheet is made up of one or more style instructions (called rules or rule sets) that describe how an element or group of elements should be displayed.
- In CSS, the two main sections of a rule are selectors and declarations



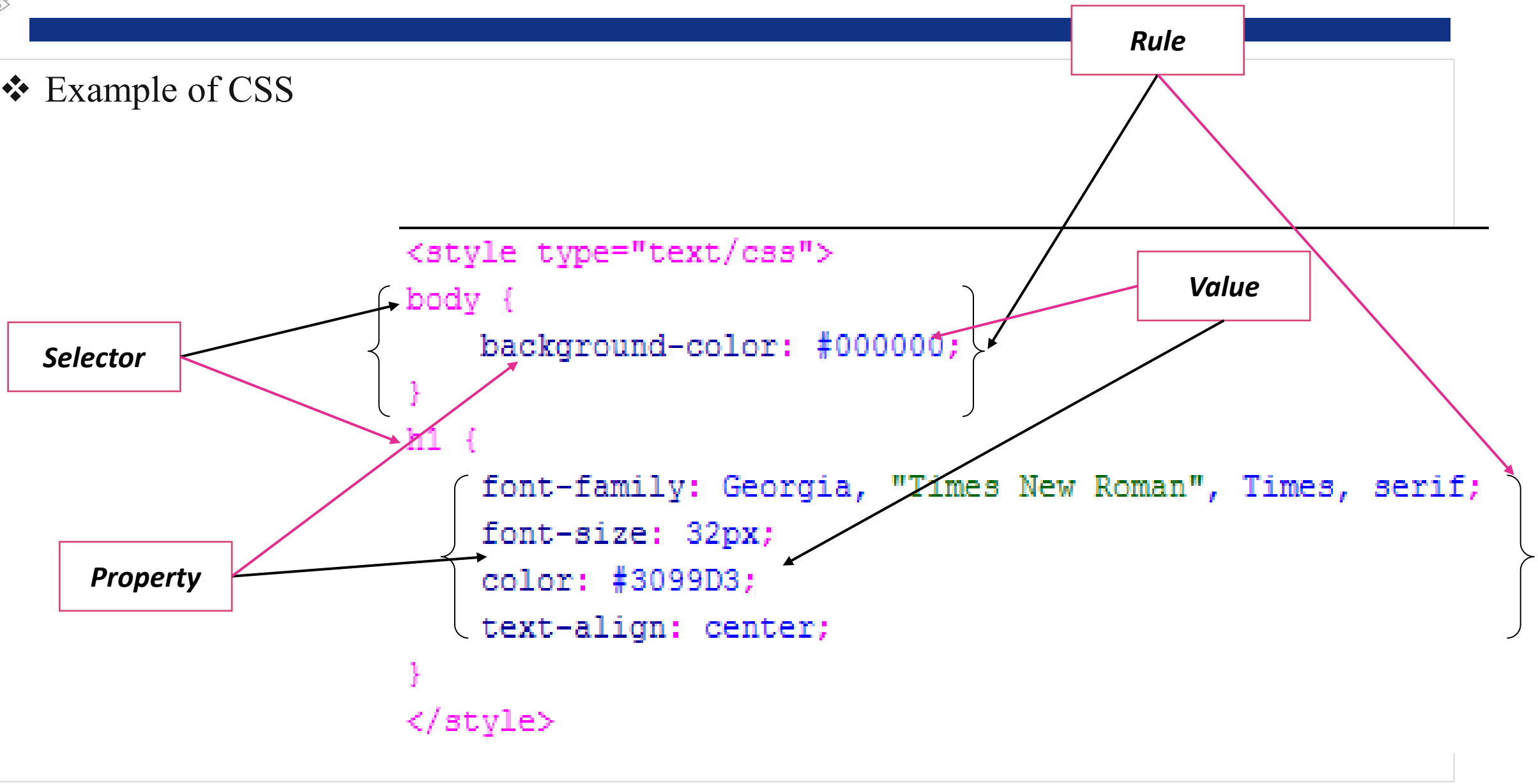
Example: You can define a table border as follows:

```
table{
        border : 1px solid #C00;
}
```

Here table is a selector and border is a property and the given value 1px solid #C00 is the value of that property.

Cont...

- Example of CSS



Cont...

❖ Selectors

- Selector identifies the element or elements to be affected. This is called an element type selector, and it is the most basic type of selector.
- For example, if use the h1 and p elements as selectors:

```
h1 { color: green; }

p { font-size: small; font-family: sans-serif; }
```

- The properties defined for each rule will apply to every h1 and p element in the document.
  - ➢ This makes all the h1 elements in the document green and specifies the paragraphs in a small, sans-serif font.

universal selector

type selector with one element

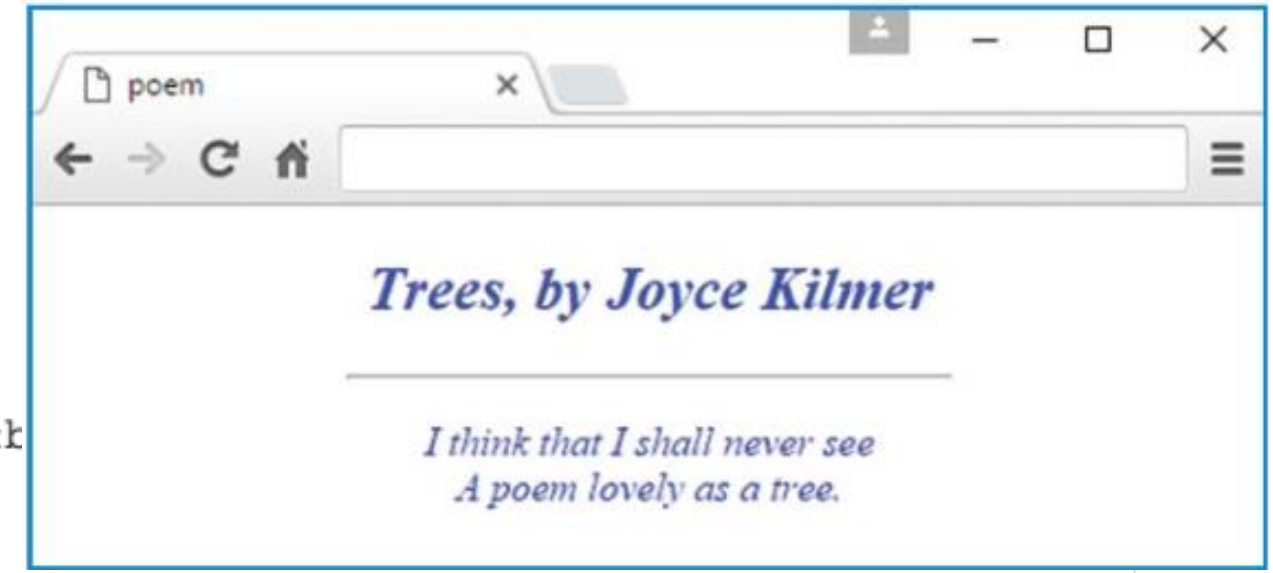
type selector with comma-separated list of elements

CSS values

CSS properties

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="author" content="John Dean">
<title>poem</title>
<style>
  * {text-align: center;}
  hr {width: 50%;}
  h2, p {font-style: italic; color: blue;}
</style>
</head>

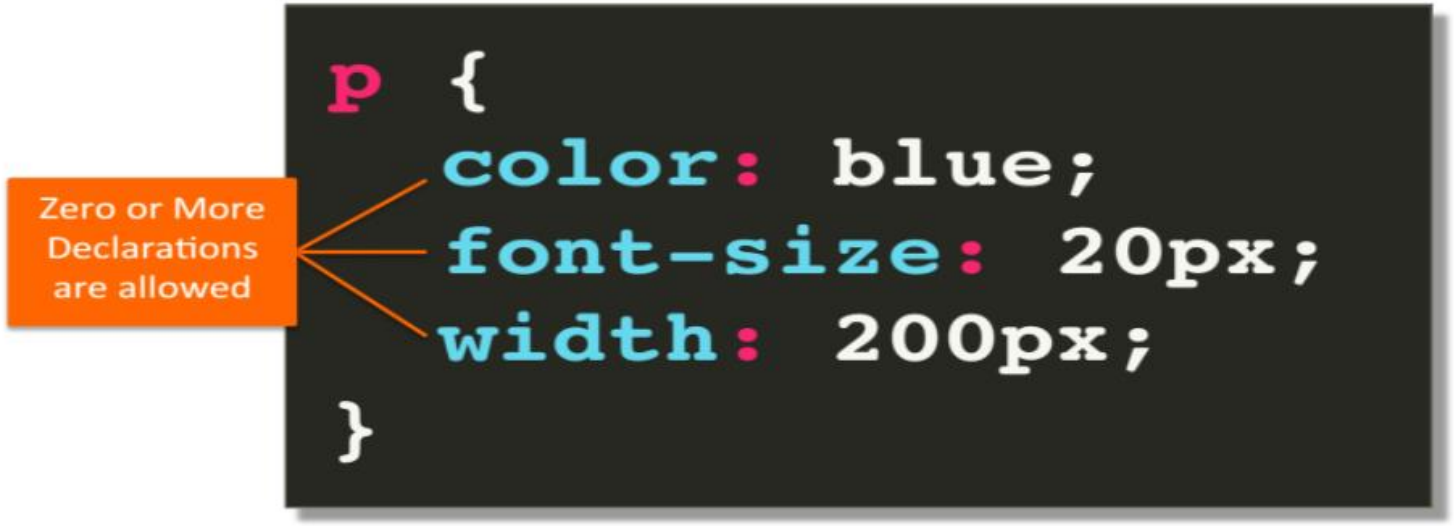
<body>
<h2>Trees, by Joyce Kilmer</h2>
<hr>
<p>
  I think that I shall never see<b
  A poem lovely as a tree.
</p>
</body>
</html>
```



Cont...

❖ Declarations

- The declaration is made up of a **property** (such as color) and its **value** (green), separated by a **colon and a space**.
  - ➢ If the semicolon is omitted, the declaration and the one following it will be ignored.
- There can be more than one declaration in a single rule are **placed inside curly brace**.



Basic CSS Selectors

❖ There are many types of selectors in CSS,

- **Type/element selector:** uses an element type (e.g., h1) to match all instances of that element type and then apply specified formatting features to those instances.
  - ➢E.g. 'p', 'div', and 'h1' etc.
- **Universal selector:** uses the same syntax as the type selector, except that instead of specifying an element type, you specify ***.** the *** matches** every element in a web page's collection of elements

```
* {text-align: center;}
```

- **Class selector:** The dot thing (**.red** in this example) is called a class selector because its purpose is to select elements that have a particular value for their class attribute.

  e.g. **'.h3_blue'**

```
.red {background-color: tomato;}
```

class selector

Cont...

- **As with type selectors,** you can have more than one class selector share one CSS rule.
  - Just separate the selectors with commas and spaces, like this:

```
.red, .h3_blue, h1 {background-color: tomato;}
```

  - With a type selector, your selector name (h1 in the this example) comes from the set of predefined HTML element names. But for a class selector, you make up the selector name.

➢ **When you make up the selector name, make it descriptive**

Class Selectors with element type prefixes

- A **class selector with an element type prefix**. Here's the syntax:

  element-type.class-value {property1: value; property2: value;}

- And here's an example CSS rule that uses a class selector with an element type prefix:

```
p.blue {background-color: skyblue;}
```

  - It is a standard class selector in that it **looks for class="blue" only in p elements.**

Class Selectors with * prefixes

- Instead of prefacing a class selector with an element type, as an alternative, **you can preface a class selector with an *.**
  - Because *** is the universal selector**, it matches all elements.
  - **Therefore, the following CSS rule is equivalent to a standard class selector rule (with no prefix):**

    ***.class-value {property1: value; property2: value;}**

  - So what would the following CSS rule do?

```
*.big-warning {font-size: x-large; color: red;}
```

    - It would match all elements in the web page that **have a class attribute value** of **big-warning.**

- **Note that: The HTML5 standard does not allow spaces within class attribute values, so it would have been illegal to use *.big warning.**

Cont...

- **ID selector**: An ID selector is similar to a class selector in that it relies on an element attribute value in searching for element matches.
  - an ID selector uses an element's **id attribute** (as opposed to a class selector, which uses an element's class attribute).
- A significant feature of an **id attribute** is that **its value must be unique within a particular web page.**
  - That's different from a **class attribute's** value, which **does not have to be unique within a particular web page.**

#id-value {*property1*: *value*; *property2*: *value*;}

ID selector

```
#lizards-lounge {color: green;}
```

Cont...

❖ The id Selector

- The id selector is used to specify a style for a single, unique element.
- The id selector uses the id attribute of the HTML element, and is defined with a "#".
- The style rule below will be applied to the element with **id="para1"**

```
#para1
{
 text-align:center;
 color:red;
}
```

```
<style type="text/css">
   #menubar {
   font-size: 12pt;
   font-family: "Luxi Serif", serif;
   }
</style>
   <ul id="menubar" >
      <li>Item 1</li> ...
   </ul>
```

Cont...

❖ The class Selector

- The class selector is used to specify a style for a group of elements. Unlike the id selector, the class selector is most often used on several elements.
- This allows you to set a particular style for any HTML elements with the same class.
- The class selector uses the HTML class attribute, and is defined with a "."
- In the example below, all HTML elements with class="center" will be center aligned:

Example

```
.center
{
   text-align: center;
}
```

```
<style type="text/css">
   p.blog {
        font-size: 12pt;
        font-family: "Luxi Serif", serif;
          }
</style>
   <p class="blog"> Today was the best day
   ever ... </p>
```

Cont...

```html
<!-- css.html -->

<!DOCTYPE html>
<html>
<head>
    <title>CSS Selectors</title>
    <link rel="stylesheet" type="text/css" href="asset/css/my_css.css">
</head>
<body>
    <h3>CSS Selectors</h3>

    <p class='c_head'> Paragraph with class 'c_head' </p>
    <p id='i_head'> Paragraph with id 'i_head' </p>

</body>
</html>
```

```css
/* asset/css/my_css.css */

/*element selection*/
h3 {
    color: blue;
}


/*class selection*/
.c_head{
    font-family: cursive;
    color: orange;
}


/*id selection*/
#i_head{
    font-variant: small-caps;
    color: red;
}
```

More selectors

Selectors	Description
h1, p, span etc.	element selector
.className	class selector
#idName	id selector
*	Universal selector (selects everything)
h1.className	select h1 with class 'className'
h1#idName	select h1 with id 'idName'
p span	descendant selector (select span which is inside p)
p > span	child selector ('span' which is direct descendant of 'p')
h1, h2, p	group selection (select h1, h2 and p)
span[my_id]	select 'span' with attribute 'my_id'
span[my_id=m_span]	select 'span' with attribute 'my_id=m_span'

Grouped Selectors

- If you ever need to apply the same style property to a number of elements, you can group the selectors into one rule.
- This one rule has the same effect as the five rules listed separately.

```
h1, h2, p, div, img { border: 1px solid blue; }
```

- Grouping them makes future edits more efficient and results in a smaller file size.

Cont...

❖ **Note that:** there are two class selectors for the class attribute's value—white and orange-background.

```
<span class="white orange-background">$10</span>
```

- That means that both the white and orange-background CSS rules get applied to the span element's content.

➢**Note that the two class selectors are separated with spaces.**

- The delimiter spaces are required whenever you have multiple class selectors for one class attribute.

Cont...

- Comments are used to explain your code, and may help you when you edit the source code at a later date.
- CSS comments are ignored by browsers.
- A CSS comment begins with /* and ends with */

```
/*This is a comment*/
p{
  text-align:center;
  /*This is another comment*/
  font-family:arial;
}
```

Linking CSS to HTML

- It is possible to link CSS with your HTML pages in two different ways:
  - Internal style, and
  - External style.
- Internal CSS can be either
  - Inline or
  - Embedded

I. Creating an Inline Style

- You can apply styles to a single element using the **style attribute in the element itself.**
- Inline styles have the structure:

  <tag style=" property: value">
- Example:

```
<h1 style="color: red">Introduction</h1>
```

- **The style attribute is a *global attribute*, which means it can be used with any element.**

Inline style

```
<!-- css.html -->

<!DOCTYPE html>
<html>
<head>
    <title>CSS Tutorial</title>

</head>
<body>

    <h3 style="color:blue"> Heading 1 </h3>
    <h3 style="color:blue"> Heading 3 </h3>
    <h3 style="color:blue"> Heading 3 </h3>

</body>
</html>
```

II. Creating Embedded Styles

- We use the **<style> tag** head section. ***<style> tag*** has a property called *type* which should be set to ***text/css*** for CSS styling.
- The format for this is shown in the example below:

```
<html>
<head> <title>Styling with CSS<title>
   <style type="text/css">
         /*CSS Content Goes Here*/
         p { color: red; text-align: center; }
   </style>
</head>
```

```
<!-- css.html -->

<!DOCTYPE html>
<html>
<head>
    <title>CSS Tutorial</title>

    <style type="text/css">
        h3.h3_blue{      /*change color to blue*/
            color: blue;
        }

        h3.h3_red{       /*change color to red*/
            color:red;
        }
    </style>

</head>
<body>

    <h3 class='h3_blue'> Heading 1 </h3>
    <h3 class='h3_blue'> Heading 3 </h3>
    <h3 class='h3_blue'> Heading 3 </h3>


    <h3 class='h3_red'> Heading 1 </h3>
    <h3 class='h3_red'> Heading 3 </h3>
    <h3 class='h3_red'> Heading 3 </h3>

</body>
</html>
```

Cont...

- In the embedded method each HTML file contains the CSS code needed to style the page.
  - This method can be good if you need to **style only one page,** or you want different pages to have varying styles.

III. Creating an External Style Sheet

- An external CSS file can be created with any text editor such as *Notepad* or *Notepad++*.
  - A CSS file contains no HTML, only CSS, and is saved with the **.css file extension.**
- You can link to the file externally by placing one of the following links in the head section of the HTML file.

```
<link rel="stylesheet" href="filename.css"/>

<style type="text/css">@import
 url("filename.css")</style>
```

Cont...

- **Note that** the rel="stylesheet" attribute-value pair. rel stands for "relationship," and its value tells the browser engine what to do with the href file.
  - Having a **rel value of stylesheet** tells the browser engine to look for CSS rules in the **href file and apply them to the current web page.**
- Why is sharing an external CSS file helpful?
  - With a shared external CSS file, **it's easy to ensure that all the web pages on your site follow the same common CSS rules.**
  - And **if you want to change those rules, you change them in one place**, in the external file, and the change affects all the web pages that share the external file

Example External

```html
<!DOCTYPE html>
<html>
<head>
    <title>CSS Tutorial</title>
    <link rel="stylesheet" type="text/css" href="asset/css/my_css.css">
</head>
<body>

    <h3 class='h3_blue'> Heading 1 </h3>
    <h3 class='h3_blue'> Heading 3 </h3>
    <h3 class='h3_blue'> Heading 3 </h3>


    <h3 class='h3_red'> Heading 1 </h3>
    <h3 class='h3_red'> Heading 3 </h3>
    <h3 class='h3_red'> Heading 3 </h3>

</body>
</html>
```

```css
/* asset/css/my_css.css */

h3.h3_blue{
    color: blue;
}

h3.h3_red{
    color:red;
}
```

Cont...

Places Where CSS Rules Can Be Defined, Highest to Lowest Priority
1. In an element's `style` attribute.
2. In a `style` element in the web page's head section.
3. In an external file.
4. In the settings defined by a user for a particular browser installation.
5. In the browser's native default settings.

- **Precedence**
  - **Local > Internal > External**
- **Selector Priority level :**
  - **ID (highest priority)**
  - **Class**
  - **Element**

Styling HTML with CSS

- Color properties
- Style Background Properties
- Styling Text properties
- Styling Fonts properties
- Styling Links properties
- Styling Lists properties
- Styling Tables properties

1. Styling Color Properties

- Two color properties—color and background-color.
  - The color property specifies the color of an element's text.
  - The background-color property specifies the background color of an element.
- You can specify a color value using one of different formats
  - Color name—for example, red
  - RGB value—specifies amounts of red, green, and blue
  - RGBA value—specifies red, green, and blue, plus amount of opacity
  - HSL value—specifies amounts of hue, saturation, and lightness
  - HSLA value—specifies hue, saturation, and lightness, plus amount of opacity.

Color Properties

- **Color name**
  - `.roofColor { color: darkred; }`
- **RGB Values for Color**
  - percentage—0% to 100% for each color
  - integer—0 to 255 for each color
    - `.eggplant {color: rgb(52%,20%,45%);}`
    - `.black {color: rgb(0%,0%,0%);}`
    - `.white {color: rgb(100%,100%,100%);}`
    - `.favorite1 {color: rgb(144,238,144);}`
    - `.favorite2 {color: rgb(127,127,127);}`

2. Styling Backgrounds

Background Color

❖ You can specifically declare a color for the background of an element.

background-color: value;

❖ Values:

- color name (e.g. red, green, blue, etc.)
- hexadecimal number (e.g. #ff0000, #00ff00, #0000ff, etc)
- RGB color code (e.g. rgb(255,0,0), rgb(0, 255, 0), rgb(0,0,255), etc)
- Transparent

❖ Example:

```
h1{background-color: rgb(255, 255, 0);}
p{background-color: #0000FF; }
```

Background Image

❖ You can set an image for the background of an element .

background-image: url(path_to_image);

Styling Backgrounds…

Background Attachment

- You can set whether the background scrolls with the page or is fixed when the user scrolls down the page.
  - This is done with the background-attachment property

    background-attachment: value;

- Values:
  - fixed
  - scroll

Background Repeat

- You can set if an image set as a background of an element is to repeat (across=x and/or down=y) the screen.

  background-repeat: no-repeat | repeat | repeat-x | repeat-y;

Styling Backgrounds…

Background Position

- You can position an image used for the background of an element using the background-position property.

```
background-position: value;
```

- Values:
  - top left
  - top center
  - top right
  - center left
  - center center
  - center right
  - bottom left
  - bottom center
  - bottom right
  - x-% y-%
  - x-pos y-pos
- Example

```
p {  background-image: url("never.jpg");
     background-position: top right;  }
```

Styling Backgrounds…

Shorthand Form

- To shorten the code, it is also possible to specify all the background properties in one single property.
- This is called a shorthand property.
- The shorthand property for background is background.

  background: color image position repeat attachment;

- Example:

```
h1{background: url("never.jpg") top left repeat fixed;}
```

Styling Backgrounds…

❖ Example

```
body {
        background-image: url("never.jpg");
        background-position: top right;
        background-repeat: repeat;
        background-attachment: fixed;
  }
```

3. Styling Text

Color

❖ You can set the color of text with the following:

```
color: value;
```

❖ Possible values are:

- color name – example: red, black…
- hexadecimal number – example: #ff0000, #000000
- RGB color code – example: rgb(255, 0, 0), rgb(0, 0, 0)

❖ Example:

```
p {  color: blue;  }
```

Styling Text…

Letter Spacing

- You can **adjust the space between letters using letter-spacing.**
- Property sets the horizontal spacing behavior between text characters.
- Positive values of letter-spacing **causes characters to spread farther apart**, while negative values of letter-spacing **bring characters closer together.**
  - You can use **negative values** which make the **letter overlap each other.**

    letter-spacing: value;
- Possible values are
  - Normal: The normal letter spacing for the current font.
  - Length: Specifies extra inter-character space *in addition to* the default space between characters.

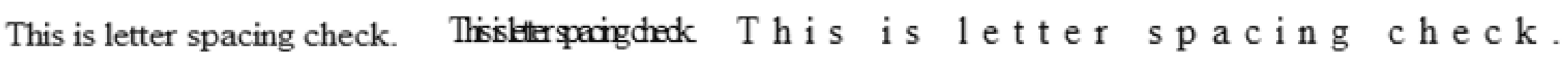


Fig. different letter spacing (0 i.e. normal, -2, 6)

Styling Text…

- A large positive or negative letter-spacing value will make the word(s) **the styling is applied to unreadable.**
  - For text styled with a **very large positive value**, the letters will be so far apart that the word(s) will appear **like a series of individual, unconnected letters.**
  - For text styled with a **very large negative value**, the letters will overlap each other to the point where the word(s) **may be unrecognizable.**
- **Legible letter-spacing must be determined on a case-by-case basis, as different font families have different character widths.**
- **There is no one value that can ensure all font families automatically maintain their legibility.**

Styling Text…

Word Spacing

- You can **adjust the space between words** in the following manner.
- You can use negative values to make them overlap.

word-spacing: value;

- Possible values are
  - normal | length

```
This line has normal word-spacing

This    line    has    a    word-spacing    of    2em

This line has a word-spacing of 5px
```

Line height

- You can **set the distance between lines** in the following way:

line-height: value;

- Possible values are
  - normal | number | length | Percentage(%)

Styling Text…

- The line-height property defines the minimum distance from baseline to baseline in text.
- A baseline is the imaginary line upon which the bottoms of characters sit.
- Most browsers split the extra **space above and below the text**, thus centering it in the overall line height.

Text Align

- property sets the horizontal alignment of the inline-level content inside a block element or table-cell box.
- You can align text with the following:

  text-align: value;

- Possible values are
  - Left | right | center | Justify

Styling Text…

Text Decoration

- property sets the appearance of decorative lines on text.
- You can decorate text with the following:

  text-decoration: value;

- Possible values are
  - None | underline | overline | Line-through
- Examples:

  This text is underlined.

  ~~This text has a line through it.~~

  This text is overlined.

```
h1{
   text-decoration-line: overline;
   text-decoration-color: black;
   text-decoration-thickness: 5px;
   text-decoration-style: dashed;
   color: blue;
    }
```

Styling Text…

Text Transform

- property specifies how to capitalize an element's text. It can be used to make text appear in all-uppercase or all-lowercase, or with each word capitalized.
- You can control the **case of letters in an HTML element** with the following:

```
text-transform: value;
```

- Possible values are
  - None | capitalize | lowercase | Uppercase
- Example:

```
h3 { text-transform: uppercase; }
```

Styling Text…

Text Indent

- property sets the length of empty space (indentation) that is put before lines of text in a block.
- You can **indent the first line of text** in an HTML element with the following:

  text-indent: value;

- Possible values are
  - Length | percentage(%)
- This property indents the first line of text (e.g. paragraph) by a specified amount.
- Example: three paragraphs indented differently

```
p#1 { text-indent: 2em; }
p#2 { text-indent: 25%; }
p#3 { text-indent: 35px; }
```

4. Styling Fonts

Font-Family

- property specifies a **prioritized list of one or more font family names** and/or **generic family names for the selected element.**
- You can set **what font will be displayed** in an element with the **font-family** property.
- There are two choices for values:
  - family-name | generic family
- Since we **can't guarantee the availability of the fonts we use on webpages**, you can supply a font stack so that the browser has multiple fonts it can choose from.
  - This simply involves a font-family value consisting of multiple font names **separated by commas.**

p{ font-family: “Times New Roman”, Georgia, Serif;}

- In such a case, the **browser starts at the beginning** of the list and looks to see if that **font is available on the machine**. If it is, it applies that font to the selected elements. **If not**, it moves on to the next font, and so on.

Styling Fonts

- It is a good idea to provide a generic font name at the end so that if none of the listed fonts are available, the browser can at least provide something approximately suitable.
- CSS defines five generic names for fonts:

  serif | sans-serif | monospace | cursive | fantasy

- It represents a worst-case scenario where the browser will try to do its best to provide at least a font that looks appropriate.
- Example:

```
body { font-family: Verdana, sans-serif; }
```

Styling Fonts…

Font Size

- You can **set the size of the text used** in an element by using the **font-size property**.

  font-size: value;

- The value can be:
  - absolute size | relative size | length | percentage(%)
- Absolute sizes are:
  - xx-small | x-small | small | medium | large | x-large | xx-large
- Absolute-size mapping to HTML heading and absolute font-sizes

CSS absolute-size values	xx-small	x-small	small	medium	large	x-large	xx-large
HTML font sizes	1	2	3	4	5	6	7

Styling Fonts…

- Relative sizes are:
  - larger | smaller
- A relative-size is interpreted relative to the table of font sizes and the font size of the parent element.
  - For example, if parent font size = *medium*, a value of *larger* will make the font size of the current element be *large*.
- If the parent element's size is not close to a table entry, the user agent is free to interpolate between table entries or round off to the closest one.

Styling Fonts…

Length Units in CSS

- There are two types of length units:
  - Relative | absolute
- **Relative length units** specify a length relative to another length property.
  - Style sheets that use relative units can more easily scale from one output environment to another.
- The absolute units consist of the physical units (in, cm, mm, pt, pc) and the px unit:
  - **in**: inches — 1in is equal to 2.54cm.
  - **cm**: centimeters | **mm**: millimeters
  - **pt**: points — the points used by CSS are equal to 1/72 of 1 inch.
  - **pc**: pica — 1pc is equal to 12pt.
  - **px**: pixel units — 1px is equal to 0.75pt (1/96 of 1 inch)

Styling Fonts…

- Example: all the following are possible

```
h1 { line-height: 0.5in }          /* inches */
h2 { line-height: 3cm }            /* centimeters */
h3 { word-spacing: 4mm }   /* millimeters */
h4 { font-size: 12pt }   /* points */
h4 { font-size: 1pc }    /* picas */
p { font-size: 12px }    /* px */
```

Styling Fonts…

- Relative units are:
  - **em**: it is equal to **the current font-size of the element** or **the parent element**.
    - ➢It is interpreted relative to the font-size of the element or its nearest parent.
  - **rem**: rem is **interpreted relative to the font-size of root element**.
- The **em unit** is equal to value of the *font-size* property of the element on which it is used.
  - When *em* occurs in the value of the *font-size* property itself, it refers to the font size of the parent element.
- Example:

```
body {
      font-size: 12px;
      text-indent: 3em; /* i.e. 36px  - based on font-size*/
      }
```

Styling Fonts…

- *rem* values are relative to the root element (i.e. html), **not to the parent element**.
- Hence, **if font-size of the root element is 16px then 1rem=16px for all elements.**
  - **If font-size is not explicitly defined in root element** then 1rem will be equal to the default font-size provided by the browser which is usually **16px.**

```
html {
    font-size: 10px;
}
span {
    font-size: 2rem;   /* font size of this span will be 20px */
}
```

Styling Fonts…

- Example:

```
h1 { line-height: 1.2em; }
```

  - This means the line height of 'h1' elements will be 20% greater than the font size of the h1 elements.

- On the other hand:

```
h1 { font-size: 1.2em; }
```

  - This means that the font-size of h1 elements will be 20% greater than the font size inherited by h1 elements.

- What is the font size of H1?

```
body { font-size: 20pt;  }
H1 { font-size: 1.2em;   }  What is the font-size of h1
 /* The font size of H1 is 24 */
```

Styling Fonts…

- In case of ***<percentage>,*** it is interpreted in relation to the parent element's font size.
  - It is much like the ***em*** **unit** except for a few fundamental differences.
- The current font-size is taken as 100%.
- While using the percent unit, your text remains fully scalable for mobile devices and for accessibility.

```
<body style="font-size: 20pt;">
<p style="font-size: 50%;">Smaller font size – 10pt</p>
<p style="font-size: 150%">Larger font size – 30pt</p>
```

Styling Fonts…

Font Style

- You can **set the style of text** in a element with the font-style property:

  font-style: value;

- Possible values are
  - Normal | *italic* | *Oblique*
- Italic and oblique are both slanted versions of the font. The difference is that the italic version is usually a separate typeface design with curved letter forms, while oblique text takes the normal font design and just slants it. However, in most browsers, they may look exactly the same.

Font Variant

- You can set the variant of text within an element with the font-variant property:

  font-variant: value;

- Possible values are
  - Normal | small-caps | all-small-caps |

Styling Fonts…

Font Weight

- property sets the **weight (or boldness) of the font.** The weights available depend on the font-family that is currently set.
- You can control the weight of text in an element with the font-weight property:

  font-weight: value;

- Absolute values are:
  - Normal | bold | 100 | 200 | 300 | 400 | 500 | 600 | 700 | 800 | 900
- Relative Values are:
  - Lighter | bolder

100	Lightest.
200	Bolder than 100, lighter than 300.
300	Bolder than 200, lighter than 400.
400	Bolder than 300, lighter than 500. Equivalent of normal.
500	Bolder than 400, lighter than 600.
600	**Bolder than 500, lighter than 700.**
700	**Bolder than 600, lighter than 800. Equivalent of bold.**
800	**Bolder than 700, lighter than 900.**
900	**Boldest.**

Styling Fonts…

- The font-weight property selects the weight of the font.
  - The values 100 to 900 form an ordered sequence, where each number indicates a weight that is at least as dark as its predecessor.
- The **keyword normal** is synonymous with **400**, and **bold** is synonymous with **700**.
  - The number-based font weight is an interesting idea, but because there aren't many fonts with that range of weights.
- Lighter: One relative font weight lighter than the parent element.
- Bolder: One relative font weight heavier than the parent element.

Example:

```
body { font-weight: 600;}
p { font-weight: bolder } /* this is based on 600 and it changes to 700*/
```

Styling Fonts…

Shorthand format Font style

- It is also possible to use shorthand format to specify all font properties at once.
- The font property can set the style, weight, variant, size, line height and font:

  font: font-style font-variant font-weight font-size/line-height font-family;

- Example:

```
font: italic normal 400 small/1.4em Verdana, sans-serif;
```

- The above would set the text of an element to an **italic style, a normal variant, a bold weight, a absolute size, a line height of 1.4em and the font family to Verdana or another sans-serif typeface.**

5. Styling Links

- User agents commonly display unvisited links differently from previously visited ones.
- CSS provides the pseudo-classes 'a:link' and 'a:visited' to distinguish them:
  - ***a:link*** **–** pseudo-class applies for links that have not yet been visited.
  - ***a:visited*** – pseudo-class applies once the link has been visited by the user.
  - ***a:hover*** – pseudo-class applies while the user designates an element with some pointing device, but does not activate it.
  - ***a:active*** – pseudo-class applies while an element is being activated by the user. For example, between the times the user presses the mouse button and releases it.
  - ***a:focus*** – pseudo-class applies while an element has the focus (accepts keyboard events or other forms of text input).

Styling Links…

- Below are the various ways you can use CSS to style links.

```
a:link {
        color: #009900;
        text-decoration: overline;
}
a:visited {color: #999999;}
a:hover {color: #333333;}
a:focus {color: #333333;}
a:active {color: #009900;}
```

Remark:

- **You must declare the *a:link* and *a:visited* before you declare *a:hover.***
- **Furthermore, you must declare *a:hover* before you can declare *a:active.***

6. Styling Lists

List Style

- You can control the **appearance of ordered and unordered lists** in one declaration with the list-style property

  list-style: image position type;

- Values:
  - Image | position | type
    - ➢Or you can control them individually

List Style Image

- You can use an image for the bullet of unordered lists with the list-style property

  list-style-image: url("path to image file");

- If you use an image, it is a good idea to declare the list-style-type also in case the user has images turned off.

Styling Lists…

List Style Position

- You can control the **position of ordered and unordered lists** with the list-style-position property

  list-style-position: value;

- Values
  - Inside | outside

List Style Type

- You can control the type of bullet ordered and unordered lists use with the list-style-type property

  list-style-type: value;

- Values
  - None | disc | circle | square | decimal | decimal-leading-zero | lower-roman | upper-roman | lower-alpha | upper-alpha | lower-greek | lower-latin | upper-latin | Armenian | georgian

Styling Lists…

- list item one
- list item two
- list item three
- list item four

- Example of list style position inside
- Mark is part of the text block; the text follows immediately after the marker. No space between the marker and the text.

- list item one
- list item two
- list item three
- list item four

- Example of list style position outside
- Mark is outside the text there is space between the marker and the text.

7. CSS Box Model

- In CSS, the term *box model* is used when **talking about design and layout.**
- The CSS box model is essentially a box that wraps around HTML elements.
- Every box is composed of four parts (or *areas):*
  - margins
  - borders
  - padding, and
  - Content



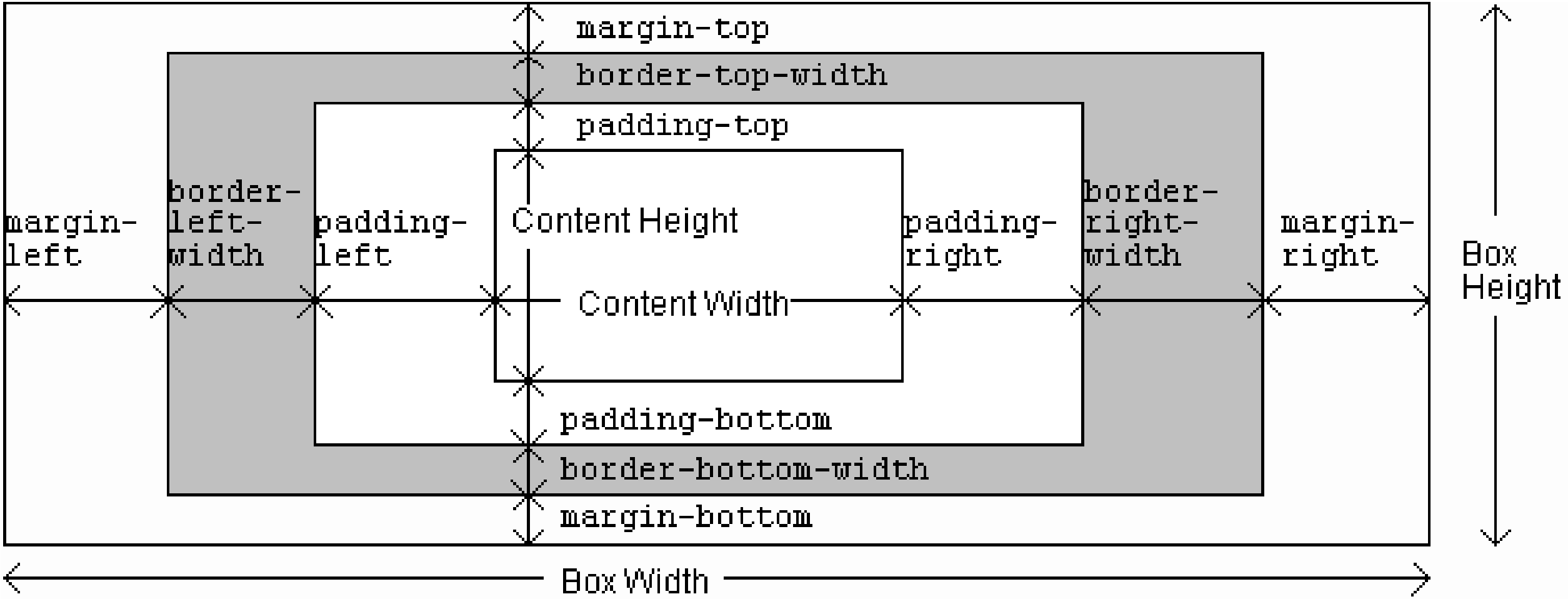
- The box model allows us to place a border around elements and space elements in relation to other elements.

CSS Box Model

Explanation of the different parts:

- **Margin** - the margin is the outermost layer which wraps the content, padding, and border as whitespace between **this box and other elements**.
  - It clears an area around the border. **The margin does not have a background color**, and it is completely transparent.
- **Border** - a border lies around the padding and content. The border-box wraps the content and any padding. **It is affected by the background color of the box.**
- **Padding** - clears an area around the content. The padding **sits around the content as white space.** The **padding is affected by the background color of the box.**
- **Content** - the content of the box, where text and images appear.

CSS Box Model



CSS Box Model: CSS Margin

- **The margin clears an area around an element outside the border.**
- The **margin does not have a background color**, and is completely transparent.

Property	Description	Values
Margin	A shorthand property for setting the margin properties in one declaration	margin-top, margin-right margin-bottom, margin-left
margin-bottom	Sets the bottom margin of an element	auto| length | %
margin-left	Sets the left margin of an element	auto| length | %
margin-right	Sets the right margin of an element	auto| length | %
margin-top	Sets the top margin of an element	auto| length | %

```
margin: 4em;
margin-top: 2em; margin-right: 250px; margin-bottom: 1em; margin-left: 4em;
```

CSS Box Model: CSS Border

Attribute	Description
Border Color	**border-color: value;** Possible value is : color name, hexadecimal number, RGB color code
Border Style	**border-style: value;** Value: dashed, dotted, double, inset, none, outset, solid
Border Width	**border-width: value;** Possible values of border width: Length, Thin, Medium, Thick
Border	**border: width, style, color;** values: width, style color. ***Border Bottom, left, right, top,*** ***border: 1px solid #333333;***

Border Bottom

- You can set the color, style and width of the bottom border around an element in one declaration with the border-bottom property.

```
border-bottom: 1px solid #333333;
```

- Values:
  - Width | style | color
- Or you can set each value individually
- [border-left] | [border-right] | [border-top]: 1px solid #333333;

CSS Box Model…

dotted: Defines a dotted border

dashed: Defines a dashed border

solid: Defines a solid border

double: Defines two borders. The width of the two borders are the same as the border-width value

groove: Defines a 3D grooved border. The effect depends on the border-color value

ridge: Defines a 3D ridged border. The effect depends on the border-color value

inset: Defines a 3D inset border. The effect depends on the border-color value

outset: Defines a 3D outset border. The effect depends on the border-color value

CSS Box Model: CSS Padding

- The padding clears an area around the content inside the border.
- The **padding is affected by the background color of the element.**
  - The **top, right, bottom, and left padding** can be changed independently using separate properties.
- A shorthand padding property can also be used, to change all padding at once.
- Syntax:

padding: length | percentage | **auto** | **inherit**

Examples:

```
padding: 10px;          /* Applied to all sides. */
padding: 10px 6px;  /* First is top & bottom, second is left & right. */
padding: 10px 20px 6px;  /* First is top, second is left & right, third is bottom */
```

Property	Description	Values
Padding	A shorthand property for setting all the padding properties in one declaration	padding-top padding-right padding-bottom padding-left
padding-bottom	Sets the bottom padding of an element	length | %
padding-left	Sets the left padding of an element	length | %
padding-right	Sets the right padding of an element	length | %
padding-top	Sets the top padding of an element	length | %

padding-top: 1em; padding-right: 3em; padding-bottom: 1em; padding-left: 3em;

8. Styling Tables

Table Borders

- To specify table borders in CSS, use the border property.
- The example below specifies a black border for table, th, and td elements:

```
table, th, td{
  border: 1px solid black;
}
```

- Notice that the table in the example above has double borders because both the table, th, and td elements have separate borders.

Styling Tables

Attribute	Description
Empty Cells	borders and backgrounds around cells that have no visible content. empty-cells: **show** | **hide**;
Table Borders	To specify table borders in CSS, use the border property **border: 1px solid black;**
Collapse Borders	table borders are collapsed into a single border or separated. border-collapse: **collapse** | **separate**;
Border Spacing	The lengths specify the distance that separates adjoining cell borders. border-spacing: *length*;
Table Width and Height	Width and height of a table are defined by the width and height properties width: length; and height: length;

Cont...

```
table {
border-collapse: separate;
border-spacing: 15px 3px;
border: none;  /* no border around the table itself */
  }
td {
border: 2px solid purple; /* borders around the cells */
  }
```

```
table{
        width: 100%;
}
th{
                height: 50px;
}
```

9. CSS Display and Visibility

- The **display property specifies if/how an element is displayed**.
  - The syntax is as follows: display: none | block | inline;
- **The visibility property specifies if an element should be visible or hidden.**
  - Syntax is as follows: visibility: hidden | visible;

Attribute	Description
none	A **none hides an element**, and it will **not take up any space**
block	A block element is an element that **takes up the full width available** and has a **line break before and after it.**
inline	An inline element **only takes up as much width** as necessary and **does not force line breaks**
hidden	hidden hides an element, but it will **still take up the same space as before**. The element will be hidden, but still affect the layout
visible	Default value. Element is visible and will occupy space in the layout.

CSS Display and Visibility…

Float

- With float property, an element can be pushed to the **left or right**, **allowing other elements to wrap around it.**
- Float is very often used for images, but it is also useful when working with layouts.
  - The syntax is:

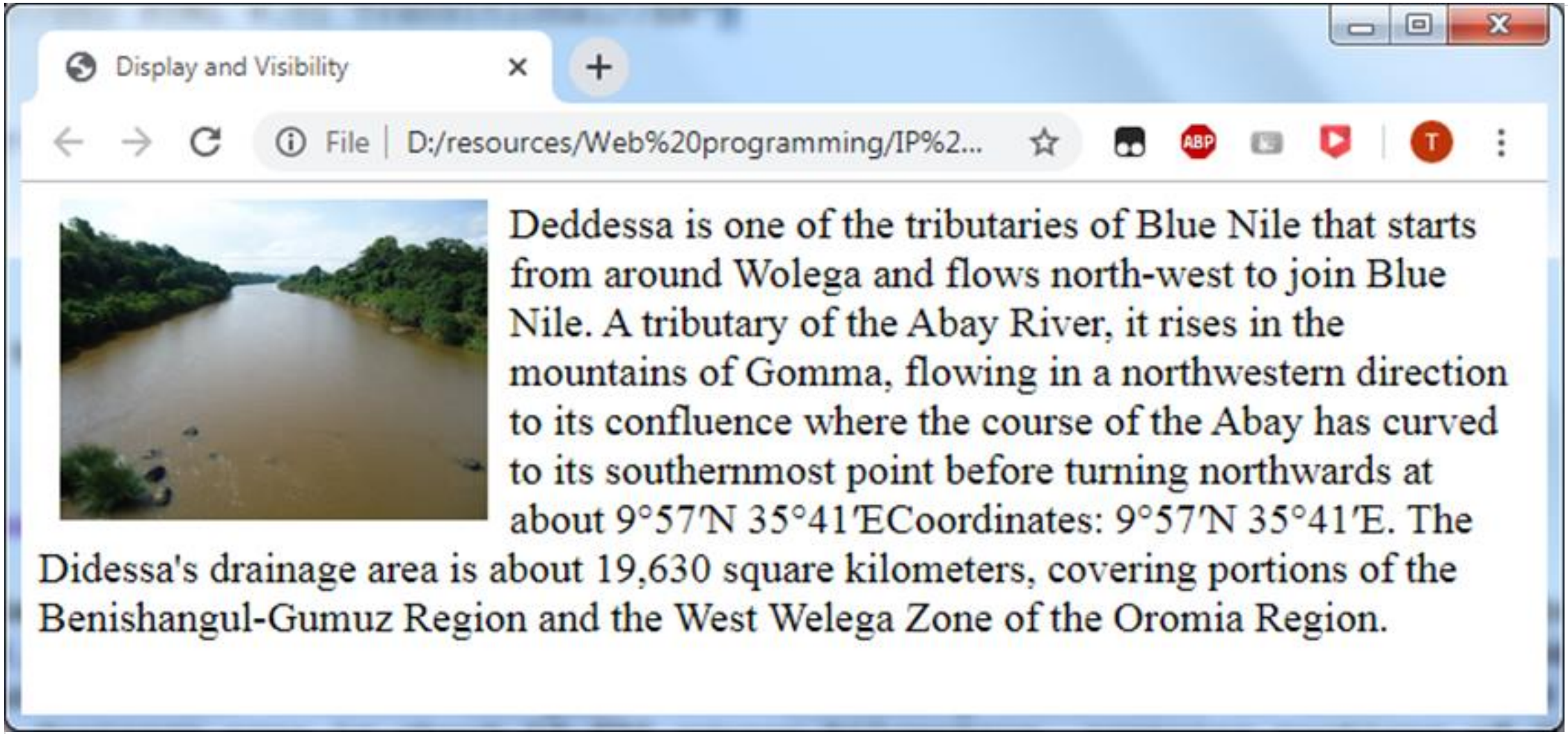
    float: left | right | none;

Attribute	Description
left	the element generates a block box that is floated to the left. Content flows on the right side of the box, starting at the top.
right	Similar to 'left', except the box is floated to the right, and content flows on the left side of the box, starting at the top.
none	The box is not floated.

CSS Display and Visibility…

```
<html>
<head>
<title>Display and Visibility</title>
<style type="text/css">
    p { font-size: 24; color: rgb(0,0,0); }
    . deddessa { float: left; }
</style>
</head>
<body>
<img class="deddessa" src="didessa.png">
<p> Deddessa is one of the tributaries of Blue Nile that starts from around Wolega and flows west to join Blue Nile. It starts from highlands of Wolega and it is joined by many streams to form a big river before finally emptying into Blue Nile. </p>
</body>
</html>
```

Cont...


Display and Visibility
File | D:/resources/Web%20programming/IP%2...
Deddessa is one of the tributaries of Blue Nile that starts from around Wolega and flows north-west to join Blue Nile. A tributary of the Abay River, it rises in the mountains of Gomma, flowing in a northwestern direction to its confluence where the course of the Abay has curved to its southernmost point before turning northwards at about 9°57′N 35°41′ECoordinates: 9°57′N 35°41′E. The Didessa's drainage area is about 19,630 square kilometers, covering portions of the Benishangul-Gumuz Region and the West Welega Zone of the Oromia Region.

10. CSS Positioning

Positioning Basics: is crucial for controlling the layout of element on a webpage.

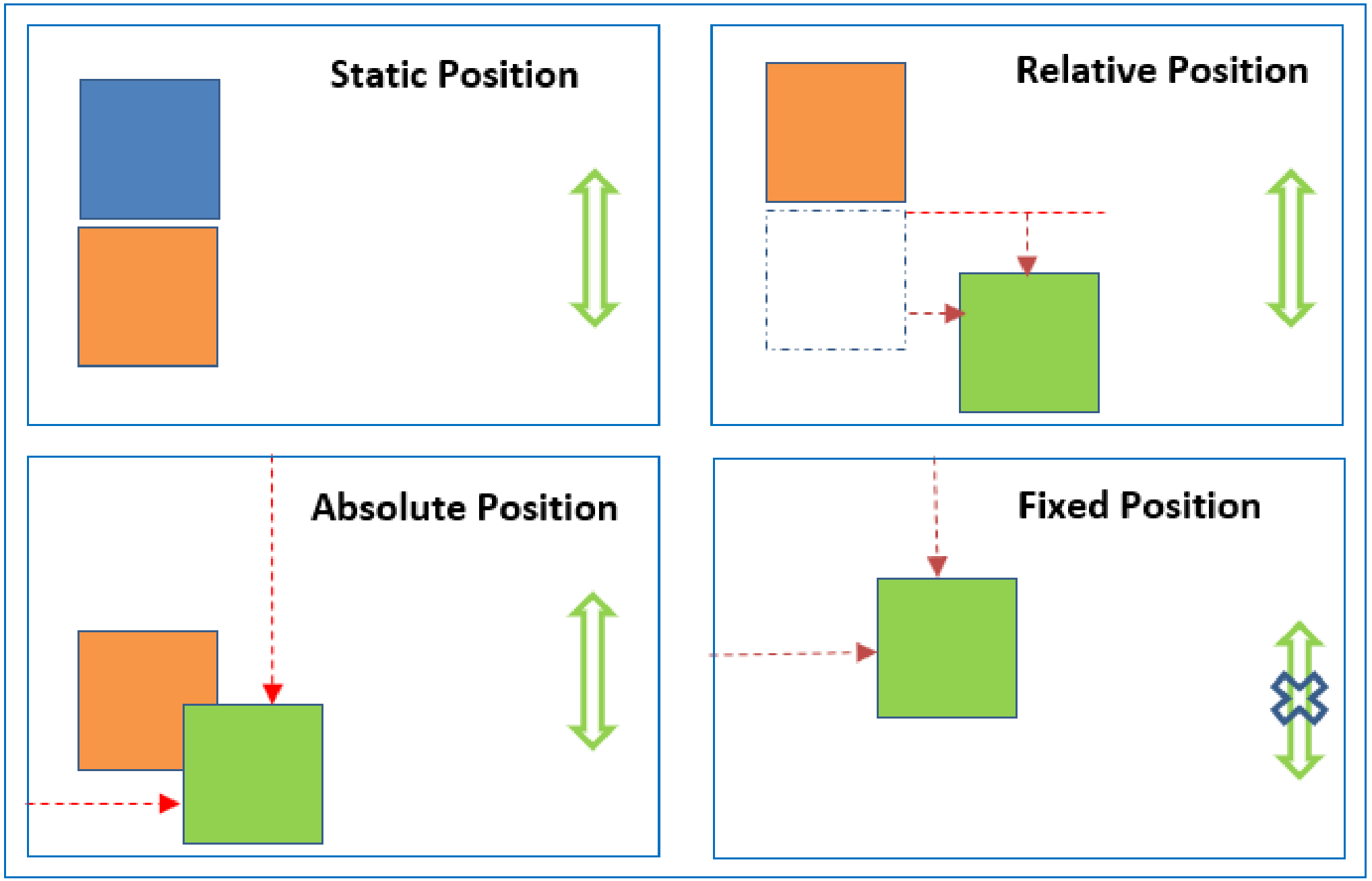
- It determine how an element is positioned in relation to its normal position,
  - Syntax:

    position: static | relative | absolute | fixed;

- **Static:** this is the normal positioning scheme in which **elements are positioned as they occur in the normal document flow.**
  - Static positioned elements are **not affected by the top, bottom, left, and right properties(used to position element in relation to their nearest positioned ancestor)**.
- **Relative:** relative positioning **moves the box relative to its original position in the flow.**
  - The distinctive behavior of relative positioning is that **the space the element would have occupied in the normal flow is preserved.**

CSS Positioning…

- **Absolute:** absolutely positioned elements are **removed from the document flow entirely and positioned relative to a containing element.**
  - Unlike relatively positioned elements, **the space they would have occupied is closed up** and hence **they have no influence on the layout of surrounding elements**.
  - An absolute position element is **positioned relative to the first parent element** that has a position other than static.
- **Fixed:** the distinguishing characteristic of fixed positioning is that **the element stays in one position in the window even when the document scrolls.**
  - Fixed elements are removed from the document flow and positioned relative to the browser window rather than another element in the document.
  - Fixed-positioned elements can overlap other elements.


Static Position
Relative Position
Absolute Position
Fixed Position

CSS Positioning…

- Once you have established the positioning method, the **actual position is specified with four offset properties**: **top, right, bottom, left**

  top: length | percentage | auto;

  right: length | percentage | auto;

  bottom: length | percentage | auto;

  left: length | percentage | auto;

- The values provided for each of the offset properties define the distance the element should be moved away from that respective edge.
- A positive value for top results in the element box moving down by that amount.
- Similarly, a positive value for the left would move the positioned element to the right by that amount

CSS Positioning…

❖Example:

```
<html>
<head>
<style type="text/css">
span {
position: relative;
top: 30px;
left: 60px;
background-color:
  fuchsia;
}
</style>
</head>
<body>
<p> Along the road he came upon a man who had <span>never worn any trousers </span>, and who was trying to put on a pair.  So he had fastened them to tree … </p>
</body>
</html>
```

CSS Positioning…

Along the road he came upon a man who had , and who was trying to put on a pair. So he had fastened them to a tree and w ight up in the air so that he should hit the two legs of the trousers as he came down. 'It would be much better if you held them in your hands,' said the young man, 'and then put your legs one after the other in each hole.'

never worn any trousers



11. CSS Grid layout

❖ **CSS Grid** is a two dimensional layout that you can use for **creating responsive items on the web.**

- Grid items are arranged in columns, and you can easily position rows without having to mess around with the HTML codes.
- A grid layout consists of a parent element, with one or more child elements.

❖ **CSS grid properties**

- **Grid container property:-** setting the container to a **display** property of ***grid*** or ***in-line grid***
  - ➢ ***display: grid***
  - ➢ ***display: in-line grid***

Cont...

- **Grid-template-column property:-** used to set each columns width.
  - It also define how many columns you want to set your project
  - grid-template-column: 100px auto 100px;
- **Grid-template-row property:-** to set height of each column.
  - It also define how many rows you want to set in your project
  - grid-template-row: 50px 50px;
- **Column-gap property:-** to set the space between two or more columns in a container.
  - Column-gap: 20px;

Cont...

- **Row-gap property:-** to set the space between two or more rows in a container
  - row-gap: 50px;
- **Note that** we can assign a gap to both the columns and rows of the container by using the gap properties
  - gap: 20px;
- **grid-column and grid-row**

Header

SignUp



Section



Article

Aside 1

Aside 2

Footer

Inheritance

- Some style sheet properties are inherited and others do not.
- In general, properties related to the **styling of text** — font size, color, style, etc — are **passed down.**
  - Properties such as **borders, margins, backgrounds, height, width**, and so on that affect the boxed area around the element are **not passed down.**
- You can use inheritance to your advantage when writing style sheets.
  - For example, **if you want all text elements to be rendered in the Verdana font face, you could write separate style rules for every element in the document and set the font-face to Verdana**.
    - A better way would be to **write a style for body element**, and let all the text elements contained in the **body** inherit that style.

Cont...

Example: All texts in the following page is displayed as red because of inheritance

```
<html><head>
<title> CSS </title>
<style type=”text/css”>
    body { color: red; }
    body { margin: 10px}
</style</head>
<body>
    <h2> Well Known Novels </h2>
    <p> Romeo and Juliet </p>
    <p> Things Fall Apart</p>
<p>Kingdom of God is Among You</p>
</body></html>
```

Conflicting styles: the Cascade

- CSS **allows you to apply several style sheets** to the same document, which means there are **bound to be conflicts.**
  - For example, what should the browser do if a document's imported style sheet says that **h1 elements should be red**, but its embedded style says **h1**s should be purple?
- Generally speaking, the **closer the style sheet is to the content**, the more weight it is given.
  - **Embedded style sheets have more weight than external style sheets.**
  - **Inline styles have more weight than embedded style sheets.**
- **To prevent a specific rule from being overridden**, you can assign its "importance" with the **!important** indicator
  - include the **!important** indicator just after the property value and before the semicolon for that rule.

```
<!DOCTYPE html>
<html><head>
<style>
#myid {
  background-color: blue;
}
.myclass {
  background-color: gray;
}
p {
  background-color: red !important;
}
</style>
</head><body>
<p>This is some text in a paragraph.</p>
<p class="myclass">This is some text in a paragraph.</p>
<p id="myid">This is some text in a paragraph.</p>
</body>
</html>
```

A

This is some text in a paragraph

This is some text in a paragraph

This is some text in a paragraph.

B

This is some text in a paragraph.

This is some text in a paragraph

This is some text in a paragraph

Conflicting styles: the Cascade…

- If there are conflicts within style rules of identical weight, **whichever one comes last in the list wins.**
- Take these three rules, for example:

```
<style type="text/css">
  p { color: red; }
  p { color: blue; }
  p { color: green; }
</style>
```

  - In this scenario, paragraph text will be green because the last rule in the style sheet overrides the earlier ones.

Reading Assignment

- CSS Transitions
- CSS Animations

The End !!